1. Strong rho, round 4: the batched inversion (2026-10-22)

Study: `research/rho_batchinv_20261022/` · **Protocol:** `PROTOCOL.md` (written after the profile and the Callgrind count, before any native timing) · **Baseline:** `412b60289` (`origin/main`) · **Host:** one x86-64 cloud VM (Intel Xeon @ 2.10 GHz, 4 vCPU), no hardware PMU · **Validation:** `benchmarks/20261022-strong-rho-batchinv/VALIDATION.json`.

The strong signed-Frobenius rho is the reference every Koblitz `vs_rho` comparison prices index calculus against. This round makes its walk step cheaper without changing any walk. At n = 41 and 53 (16 fixtures) and n = 61 (4 fixtures), every output line is identical to main's, timing and RSS aside, and every scalar is recovered. The `batch_inv` unit tests pass.

1.1. Change

`Gf2::batch_inv_clmul` is the Montgomery batch inversion of the walk's 32 lanes. A Callgrind profile of main put it at 13.6% of all instructions, about 90 per element. About 18 of those went to iterator bookkeeping: `chunks().zip().zip()` over variable-length chunks. Both passes now walk the slices as `as_chunks::<4>()` arrays, each lane a constant index, with the short tail handled separately: last forward, first backward. The same products are formed. Lane assignment cannot change a result, since each inverse is unique.

1.2. Instructions (measured, Callgrind, 4 fixtures)

n	main Ir / step	round 4 Ir / step	ratio
41	706.5	679.0	1.040
53	707.2	679.7	1.040

Totals at n = 53: 2,145,944,102 ⟶ 2,062,553,397 for 3,034,474 walk steps in both arms. The batch inversion's loops fall from 71.1 to 43.6 instructions per step. Its single inverse (25.7) and the walk body (592.4) do not change.

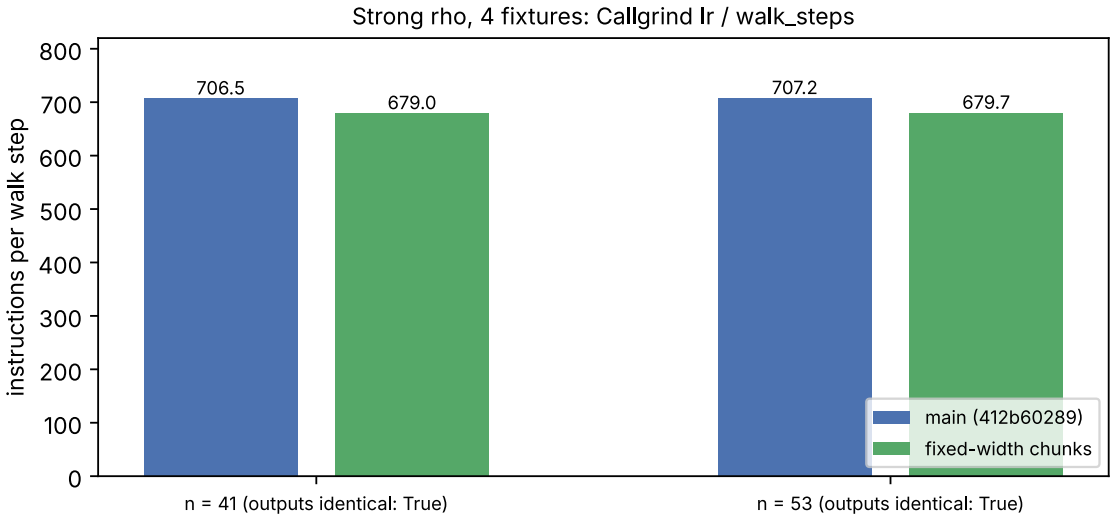


Figure 1: Instructions per walk step by arm (`runs/callgrind/`).

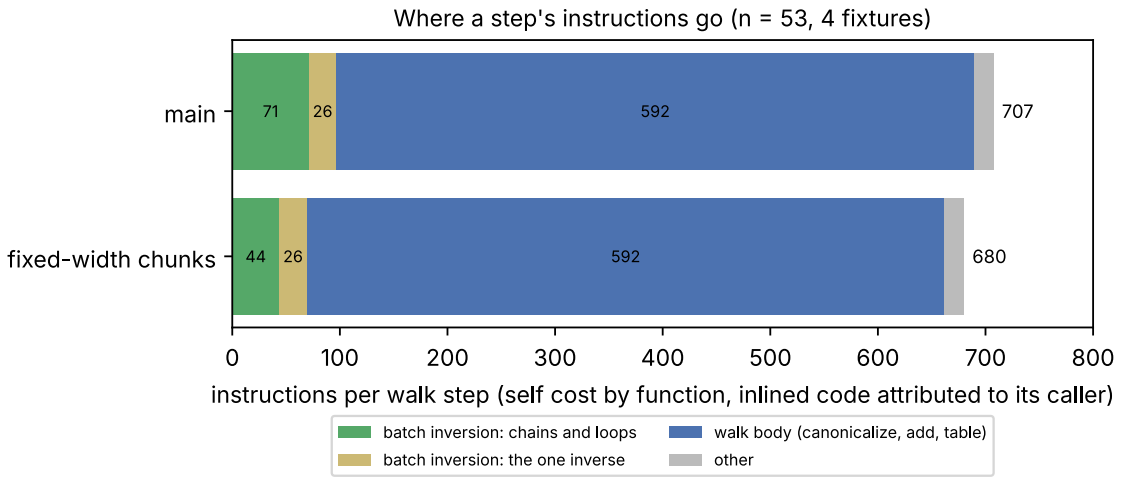


Figure 2: Where a step's instructions go at $n = 53$ (`runs/callgrind/*.annotate.txt`).

1.3. Native time (measured)

A/A (main × 6), then 5 interleaved A/B rounds: 32 processes under `tools/isolated_bench.py --cpus 3 --max-other-cpu 0.3`, none contended, all exit 0.

curve	main ns/step	round 4 ns/step	ratio	A/A spread, verdict
`icv1-f2m53-…`	110.6	99.0	1.117	7.0%: established (declared rule); ranges overlap
`icv1-f2m61-…`	117.1	108.8	1.076	14.4%: not established

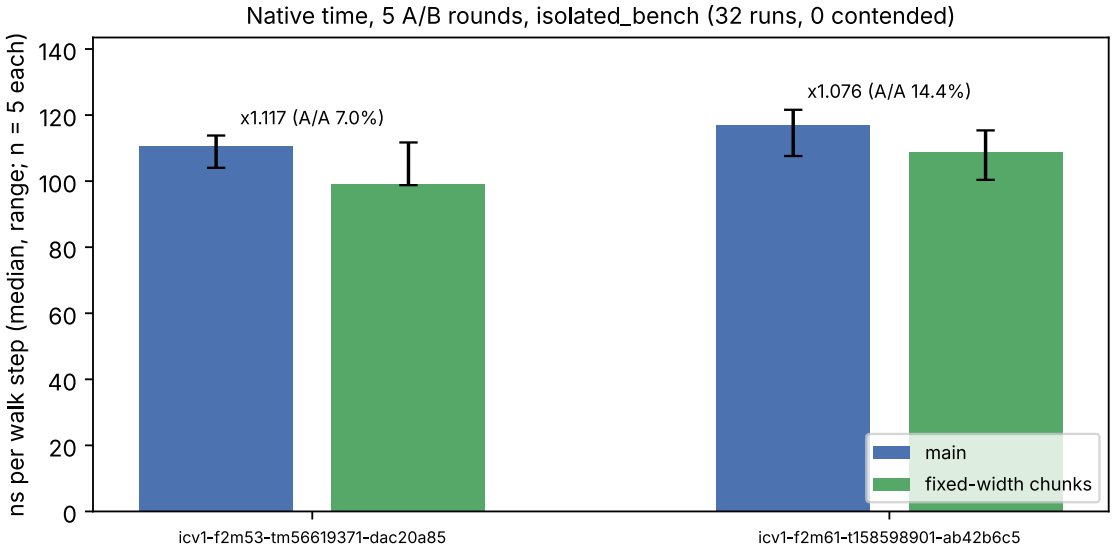


Figure 3: Native ns per walk step, median and range over 5 rounds (`runs/time/`).

The time gain at $n = 53$ is larger than the instruction gain (11.7% against 4.0%). One reading is that the old loop's bookkeeping also lengthened the dependent chains, but no hardware counter can test that here. Under the declared rule the $n = 53$ gain is established. Its five-run ranges still overlap: one candidate run, at 111.7, is slower than main's fastest, at 104.0. The $n = 61$ gain is inside its A/A spread and is not claimed.

1.4. Counted cost and the §5a band

The walks are identical, so every counted operation is too. At $n = 53$ over 16 fixtures, both arms record 8,856,125 group additions and 18,262,834 unpriced native operations (canonicalizations, partition hashes, table inserts and queries). Priced at 0, 0.1 and 1 GAE, both arms come to 8,856,125, 10,682,408.4 and 27,118,959 GAE. In counted units neither arm ranks above the other. The gain is native cost per step only.

1.5. Validation

`benchmarks/20261022-strong-rho-batchinv/` holds the gate's three items:

- a replay certificate: both binaries re-run, and every recovered scalar checked by an independent Python curve implementation (`verify_scalars.py`);
- a validator report: every headline above recomputed from `runs/`;
- the rho reference descriptor: main's strong reference, the same binary family and unit.

1.6. Scope

- **What remains.** 680 instructions per step. The walk body is 592 of them: two normal-basis round trips through byte tables, about 120, and the least-rotation search dominate. The batch inversion is 44, plus 26 for its one inverse, a serial chain of 52 carry-less squarings. Both remaining targets are named in `PROTOCOL.md`. Neither is attempted here.
- **Counted units.** GAE records do not change. Wall-time and instruction comparisons against this rho get harder for index calculus.
- **Not measured.** Hosts other than one x86-64 VM; IPC (no PMU); the $m = 83$ wide field, which uses the unchanged `WideGf2` path.